Design & Analysis of Algorithms

Lab 1: Algorithm Performance Analysis Workbench

Name: Prachi Manwal
Roll No: 2501730365
Email: 2501730365@krmu.edu.in

Q1. 1.5 Runtime and Complexity Comparison Table (5 marks)

```
import math
def generate_runtime_complexity_table(n):
 linear_count = n
 binary_count = math.floor(math.log2(n)) + 1 if n > 0 else 0
 bubble_count = n * (n - 1) // 2
 insertion_count = n * (n - 1) // 2
 return [
 "Runtime Complexity Comparison",
 "Method ObservedCount ExpectedComplexity Observation",
 f"Linear Search {linear_count} O(n) Grows linearly",
 f"Binary Search {binary_count} O(log n) Grows logarithmically",
 f"Bubble Sort {bubble_count} O(n^2) Grows quadratically",
 f"Insertion Sort {insertion_count} O(n^2) Grows quadratically"
 ]
```

Q2. 1.6 Runtime Comparison Chart Data and Scalability Report (5 marks)

```
import math

def generate_runtime_chart_report(input_sizes):
    result = [
        "Runtime Comparison Chart Data",
        "InputSize LinearSearch BinarySearch BubbleSort InsertionSort"
    ]

    for n in input_sizes:
        linear = n
        binary = math.floor(math.log2(n)) + 1 if n > 0 else 0
        bubble = n * (n - 1) // 2
        insertion = n * (n - 1) // 2

        result.append(
            f"{n} {linear} {binary} {bubble} {insertion}"
        )

    result.extend([
        "Scalability Summary",
        "Algorithm Complexity Scalability",
        "Linear Search O(n) Moderate",
        "Binary Search O(log n) Excellent",
        "Bubble Sort O(n^2) Poor",
        "Insertion Sort O(n^2) Poor",
        "Key Observations",
        "Best Algorithm: Binary Search",
        "Most Expensive Algorithm: Bubble Sort",
        "Conclusion: Logarithmic algorithms scale better for large inputs"
    ])

    return result
```